我想要自己写梯度计算、自然梯度计算函数 来进行 VMC 算法 以下是我的实现:

```
import jax
import jax.numpy as jnp
import netket as nk
import netket.experimental as nkx
import numpy as np
from pyscf import gto, scf, fci
from flax import nnx
import optax
from functools import partial
from tqdm import tqdm
import matplotlib.pyplot as plt

print(f"JAX version: {jax.__version__}")
print(f"NetKet version: {nk.__version__}")
# H₂ 分子定义
bond_length = 1.4
geometry = [('H', (0., 0., 0.)), ('H', (bond_length, 0., 0.))]
mol = gto.M(atom=geometry, basis='STO-3G', verbose=0)
mf = scf.RHF(mol).run(verbose=0)
print("H₂ HF能量:", mf.e_tot)
# FCI 精确基准
cisolver = fci.FCI(mf)
cisolver.nroots = 4
E_fcis, fcivec = cisolver.kernel()

print("="*60)
print("H₂ FCI 基准能量")
print("="*60)
for i, e in enumerate(E_fcis):
    exc = (e - E_fcis[0]) * 27.2114
    print(f"E{i} = {e:.8f} Ha  |  激发能: {exc:.4f} eV")

# NetKet 哈密顿量
ha = nkx.operator.from_pyscf_molecule(mol)

# Hilbert 空间
hi = nk.hilbert.SpinOrbitalFermions(
    n_orbitals=2,
    s=1/2,
    n_fermions_per_spin=(1,1),
)

print(f"\n希尔伯特空间大小: {hi.n_states}")
print(f"希尔伯特空间维度: {hi.size}")

class SingleStateAnsatz(nnx.Module):
    def __init__(self, n_spin_orbitals: int, hidden_dim=16, *, rngs: nnx.Rngs):
        super().__init__()
        self.linear1 = nnx.Linear(n_spin_orbitals, hidden_dim, rngs=rngs, param_dtype=complex)
        self.linear2 = nnx.Linear(hidden_dim, hidden_dim, rngs=rngs, param_dtype=complex)
        self.output = nnx.Linear(hidden_dim, 1, rngs=rngs, param_dtype=complex)

    def __call__(self, x):
        h = nnx.tanh(self.linear1(x.astype(complex)))
        h = nnx.tanh(self.linear2(h))
        out = self.output(h)
        return jnp.squeeze(out)

def forward(GraphDef_State, x):
    """NetKet 采样器需要的 forward 函数"""
    log_psi, _ = nnx.call(GraphDef_State)(x)
    return log_psi

#以上内容不允许修改

@partial(jax.jit, static_argnames=("model_forward", "graphdef"))
def compute_energy_and_gradient_vjp(graphdef, state, model_forward, hamiltonian, samples):
    """
    使用 VJP 计算能量和梯度（完全按照 NetKet 的方式）

    这与 NetKet 的 expect_forces.py 中的实现完全一致
    """
    n_samples = samples.shape[0]

    # 定义 logpsi 函数
    def logpsi_fn(s):
        return model_forward((graphdef, s), samples)

    # 计算局部能量
    log_psi = logpsi_fn(state)
    eta, H_sigmaeta = hamiltonian.get_conn_padded(samples)
```

```python
    logpsi_eta = model_forward((graphdef, state), eta)

    log_psi_expanded = jnp.expand_dims(log_psi, axis=-1)
    Eloc = jnp.sum(H_sigmaeta * jnp.exp(logpsi_eta - log_psi_expanded), axis=-1)
    energy = jnp.mean(Eloc)

    # 中心化局部能量
    Eloc_centered = Eloc - energy

    # 使用 VJP 计算梯度
    # 注意：NetKet 使用 conjugate=True
    _, vjp_fn = jax.vjp(logpsi_fn, state)

    # 计算梯度：grad = vjp(conj(Eloc_centered) / n_samples)
    grad = vjp_fn(jnp.conjugate(Eloc_centered) / n_samples)[0]

    # 乘以 2（与 NetKet 的 force_to_grad 一致）
    grad = jax.tree_map(lambda g: 2 * g, grad)

    return energy, grad

    @partial(jax.jit, static_argnames=("model_forward", "graphdef"))
def compute_qgt(model_forward, graphdef, state, samples):
    """
    计算量子几何张量（QGT）

    QGT 定义：S_{ij} = cov(O_i*, O_j)

    使用中心化的梯度计算
    """
    n_samples = samples.shape[0]

    # 定义 logpsi 函数
    def logpsi_single_param(s, x):
        return model_forward((graphdef, s), x)

    # 对每个样本计算梯度
    def grad_logpsi_single(x):
        return jax.grad(logpsi_single_param, argnums=0, holomorphic=True)(state, x)

    # 批量计算
    grads_tree = jax.vmap(grad_logpsi_single)(samples)

    # 展平梯度
    grads_flat, tree_def = jax.tree_util.tree_flatten(grads_tree)
    grads_concat = jnp.concatenate([g.reshape((n_samples, -1)) for g in grads_flat], axis=-1)

    # 计算均值和中心化
    mean_grad = jnp.mean(grads_concat, axis=0, keepdims=True)
    centered_grads = grads_concat - mean_grad

    # 计算 QGT: S = (1/N) * O^† O
    # 注意：使用中心化的梯度
    qgt = (1.0 / n_samples) * jnp.einsum('si,sj->ij', centered_grads.conj(), centered_grads)

    return qgt

from jax import flatten_util

@partial(jax.jit, static_argnames=("model_forward", "graphdef"))
def compute_natural_gradient(model_forward, graphdef, state, samples, energy_grad, epsilon=1e-4):
    """
    计算 SR 自然梯度

    自然梯度：Δθ = S^{-1} * g
    """
    # 1. 计算 QGT
    qgt = compute_qgt(model_forward, graphdef, state, samples)

    # 2. 正则化 QGT
    n_params = qgt.shape[0]
    qgt_reg = qgt + epsilon * jnp.eye(n_params)

    # 3. 展平能量梯度
    g_flat, unflatten_fn = flatten_util.ravel_pytree(energy_grad)

    # 4. 解线性方程：S * Δθ = g
    nat_g_flat = jnp.linalg.solve(qgt_reg, g_flat)

    # 5. 恢复梯度树结构
    nat_grad = unflatten_fn(nat_g_flat)
```

```python
    return nat_grad, qgt


def train_vmc_sr(
    hamiltonian,
    hilbert,
    model,
    n_iterations=300,
    n_samples=1000,
    learning_rate=0.1,
    sr_epsilon=1e-4,
    use_sr=True,
    seed=21
):
    """
    完整的 VMC + SR 训练循环
    """
    # 初始化模型
    graphdef, model_state = nnx.split(model)
    GraphDef_State = (graphdef, model_state)

    # 设置采样器
    edges = [(0, 1), (2, 3)]
    g = nk.graph.Graph(edges=edges)
    single_rule = nk.sampler.rules.FermionHopRule(hilbert=hilbert, graph=g)
    sampler = nk.sampler.MetropolisSampler(
        hilbert,
        rule=single_rule,
        n_chains=16,
        sweep_size=32
    )

    # 初始化采样器状态
    sampler_state = sampler.init_state(forward, GraphDef_State, seed=seed)

    # 设置优化器
    optimizer = optax.adam(learning_rate=learning_rate)
    opt_state = optimizer.init(model_state)

    # 记录训练过程
    energy_history = []

    print(f"\n{'='*70}")
    print(f"开始训练: {'使用 SR' if use_sr else '不使用 SR'}")
    print(f"迭代次数: {n_iterations}, 样本数: {n_samples}, 学习率: {learning_rate}")
    print(f"{'='*70}\n")

    for i in tqdm(range(n_iterations), desc="训练进度"):
        # 1. 采样
        sampler_state = sampler.reset(forward, GraphDef_State, state=sampler_state)
        samples, sampler_state = sampler.sample(forward, GraphDef_State, state=sampler_state, chain_length=n_samples
        samples = samples.reshape(-1, samples.shape[-1])

        # 2. 计算能量和梯度
        energy, grads = compute_energy_and_gradient_vjp(
            graphdef, model_state, forward, hamiltonian, samples
        )

        # 3. SR 自然梯度（如果启用）
        if use_sr:
            nat_grad, qgt = compute_natural_gradient(
                forward, graphdef, model_state, samples, grads, epsilon=sr_epsilon
            )
            update_grad = nat_grad
        else:
            update_grad = grads

        # 4. 参数更新
        updates, opt_state = optimizer.update(update_grad, opt_state, model_state)
        model_state = optax.apply_updates(model_state, updates)
        GraphDef_State = (graphdef, model_state)

        # 5. 记录
        energy_history.append(energy.real)

        # 6. 打印进度
        if i % 50 == 0:
            error = abs(energy.real - E_fcis[0])
            print(f"\nIter {i:3d} | Energy: {energy.real:.8f} Ha | Error: {error:.6f} Ha")
            if use_sr:
                print(f"         | QGT trace: {jnp.trace(qgt):.4f} | QGT condition: {jnp.linalg.cond(qgt):.2e}")

    print(f"\n{'='*70}")
    print(f"训练完成! 最终能量: {energy_history[-1]:.8f} Ha")
```

```python
    print(f"FCI 基准: {E_fcis[0]:.8f} Ha")
    print(f"误差: {abs(energy_history[-1] - E_fcis[0]):.6e} Ha")
    print(f"{'='*70}\n")

    return energy_history, model_state


# 初始化模型
rngs = nnx.Rngs(21)
model_sr = SingleStateAnsatz(n_spin_orbitals=4, hidden_dim=12, rngs=rngs)

# 训练（使用 SR）
energy_history_sr, final_state_sr = train_vmc_sr(
    hamiltonian=ha,
    hilbert=hi,
    model=model_sr,
    n_iterations=800,
    n_samples=1000,
    learning_rate=0.01,
    sr_epsilon=1e-2,
    use_sr=True,
    seed=21
)
```

1=返回的结果是：`

2 = 开始训练: 使用 SR 迭代次数: 800, 样本数: 1000, 学习率: 0.01

训练进度: 0%| | 0/800 [00:00<?, ?it/s] 训练进度: 0%| | 2/800 [00:00<00:46, 17.08it/s]

Iter 0 | Energy: -0.48420472 Ha | Error: 0.531264 Ha | QGT trace: 9.3112+0.0000j | QGT condition: 3.78e+34 训练进度: 7%|▉ | 53/800 [00:02<00:32, 22.69it/s]

Iter 50 | Energy: -0.91677998 Ha | Error: 0.098688 Ha | QGT trace: 8.1692+0.0000j | QGT condition: 8.46e+47 训练进度: 13%|▉▉ | 104/800 [00:04<00:32, 21.71it/s]

Iter 100 | Energy: -0.93989802 Ha | Error: 0.075570 Ha | QGT trace: 1.1817+0.0000j | QGT condition: 2.61e+47 训练进度: 19%|▉▉▉ | 155/800 [00:06<00:27, 23.41it/s]

Iter 150 | Energy: -0.93995927 Ha | Error: 0.075509 Ha | QGT trace: 0.4690+0.0000j | QGT condition: 6.31e+47 训练进度: 25%|▉▉▉▉ | 203/800 [00:08<00:25, 23.82it/s]

Iter 200 | Energy: -0.94062749 Ha | Error: 0.074841 Ha | QGT trace: 0.0000+0.0000j | QGT condition: 6.66e+64 训练进度: 32%|▉▉▉▉▉ | 254/800 [00:11<00:22, 23.94it/s]

Iter 250 | Energy: -0.94032412 Ha | Error: 0.075144 Ha | QGT trace: 0.0918+0.0000j | QGT condition: 2.04e+47 训练进度: 38%|▉▉▉▉▉▉ | 305/800 [00:13<00:21, 23.51it/s]

Iter 300 | Energy: -0.94012022 Ha | Error: 0.075348 Ha | QGT trace: 0.1840+0.0000j | QGT condition: 5.43e+46 训练进度: 44%|▉▉▉▉▉▉▉ | 353/800 [00:15<00:19, 22.69it/s]

Iter 350 | Energy: -0.94089373 Ha | Error: 0.074575 Ha | QGT trace: 0.0000+0.0000j | QGT condition: 6.99e+64 训练进度: 50%|▉▉▉▉▉▉▉▉ | 404/800 [00:17<00:17, 22.64it/s]

Iter 400 | Energy: -0.94093154 Ha | Error: 0.074537 Ha | QGT trace: 0.0000+0.0000j | QGT condition: 6.28e+64 训练进度: 57%|▉▉▉▉▉▉▉▉▉ | 455/800 [00:19<00:15, 22.60it/s]

Iter 450 | Energy: -0.94044417 Ha | Error: 0.075024 Ha | QGT trace: 0.2382+0.0000j | QGT condition: 4.44e+47 训练进度: 63%|▉▉▉▉▉▉▉▉▉▉ | 503/800 [00:21<00:12, 23.41it/s]

Iter 500 | Energy: -0.94094200 Ha | Error: 0.074526 Ha | QGT trace: 0.1358+0.0000j | QGT condition: 3.59e+47 训练进度: 69%|▉▉▉▉▉▉▉▉▉▉▉ | 554/800 [00:24<00:11, 22.31it/s]

Iter 550 | Energy: -0.94127106 Ha | Error: 0.074197 Ha | QGT trace: 0.0000+0.0000j | QGT condition: 9.07e+64 训练进度: 76%|▉▉▉▉▉▉▉▉▉▉▉▉ | 605/800 [00:26<00:08, 22.75it/s]

Iter 600 | Energy: -0.94128126 Ha | Error: 0.074187 Ha | QGT trace: 0.0000+0.0000j | QGT condition: 2.69e+68 训练进度: 82%|▉▉▉▉▉▉▉▉▉▉▉▉▉ | 653/800 [00:28<00:06, 23.06it/s]

Iter 650 | Energy: -0.94098489 Ha | Error: 0.074483 Ha | QGT trace: 0.1300+0.0000j | QGT condition: 6.63e+47 训练进度: 88%|▉▉▉▉▉▉▉▉▉▉▉▉▉▉ | 704/800 [00:30<00:04, 23.30it/s]

Iter 700 | Energy: -0.94130673 Ha | Error: 0.074162 Ha | QGT trace: 0.0000+0.0000j | QGT condition: 2.58e+65 训练进度: 94%|▉▉▉▉▉▉▉▉▉▉▉▉▉▉▉ | 755/800 [00:32<00:01, 23.37it/s]

Iter 750 | Energy: -0.94132098 Ha | Error: 0.074147 Ha | QGT trace: 0.0000+0.0000j | QGT condition: 1.62e+65 训练进度: 100%|▉▉▉▉▉▉▉▉▉▉▉▉▉▉▉▉| 800/800 [00:34<00:00, 23.17it/s]

3 =

==========

训练完成！ 最终能量: -0.94134622 HA FCI 基准: -1.01546825 HA 误差: 7.412202E-02 HA

4 = ==========

开始训练: 使用 SR 迭代次数: 800, 样本数: 1000, 学习率: 0.01

训练进度: 0%| | 0/800 [00:00<?, ?it/s] 训练进度: 0%| | 2/800 [00:00<00:46, 17.08it/s]

Iter 0 | Energy: -0.48420472 Ha | Error: 0.531264 Ha | QGT trace: 9.3112+0.0000j | QGT condition: 3.78e+34 训练进度: 7%|▉ | 53/800 [00:02<00:32, 22.69it/s]

Iter 50 | Energy: -0.91677998 Ha | Error: 0.098688 Ha | QGT trace: 8.1692+0.0000j | QGT condition: 8.46e+47 训练进度: 13%|▉▉ | 104/800 [00:04<00:32, 21.71it/s]

Iter 100 | Energy: -0.93989802 Ha | Error: 0.075570 Ha | QGT trace: 1.1817+0.0000j | QGT condition: 2.61e+47 训练进度: 19%|▉▉▉ | 155/800 [00:06<00:27, 23.41it/s]

Iter 150 | Energy: -0.93995927 Ha | Error: 0.075509 Ha | QGT trace: 0.4690+0.0000j | QGT condition: 6.31e+47 训练进度: 25%|▉▉▉▉ | 203/800 [00:08<00:25, 23.82it/s]

Iter 200 | Energy: -0.94062749 Ha | Error: 0.074841 Ha | QGT trace: 0.0000+0.0000j | QGT condition: 6.66e+64 训练进度: 32%|▉▉▉▉▉ | 254/800 [00:11<00:22, 23.94it/s]

Iter 250 | Energy: -0.94032412 Ha | Error: 0.075144 Ha | QGT trace: 0.0918+0.0000j | QGT condition: 2.04e+47 训练进度: 38%|▉▉▉▉▉▉ | 305/800 [00:13<00:21, 23.51it/s]

Iter 300 | Energy: -0.94012022 Ha | Error: 0.075348 Ha | QGT trace: 0.1840+0.0000j | QGT condition: 5.43e+46 训练进度: 44%|▉▉▉▉▉▉▉ | 353/800 [00:15<00:19, 22.69it/s]

Iter 350 | Energy: -0.94089373 Ha | Error: 0.074575 Ha | QGT trace: 0.0000+0.0000j | QGT condition: 6.99e+64 训练进度: 50%|▉▉▉▉▉▉▉▉ | 404/800 [00:17<00:17, 22.64it/s]

Iter 400 | Energy: -0.94093154 Ha | Error: 0.074537 Ha | QGT trace: 0.0000+0.0000j | QGT condition: 6.28e+64 训练进度: 57%|▉▉▉▉▉▉▉▉▉ | 455/800 [00:19<00:15, 22.60it/s]

Iter 450 | Energy: -0.94044417 Ha | Error: 0.075024 Ha | QGT trace: 0.2382+0.0000j | QGT condition: 4.44e+47 训练进度: 63%|▉▉▉▉▉▉▉▉▉▉ | 503/800 [00:21<00:12, 23.41it/s]

Iter 500 | Energy: -0.94094200 Ha | Error: 0.074526 Ha | QGT trace: 0.1358+0.0000j | QGT condition: 3.59e+47 训练进度: 69%|▉▉▉▉▉▉▉▉▉▉▉ | 554/800 [00:24<00:11, 22.31it/s]

Iter 550 | Energy: -0.94127106 Ha | Error: 0.074197 Ha | QGT trace: 0.0000+0.0000j | QGT condition: 9.07e+64 训练进度: 76%|▉▉▉▉▉▉▉▉▉▉▉▉ | 605/800 [00:26<00:08, 22.75it/s]

Iter 600 | Energy: -0.94128126 Ha | Error: 0.074187 Ha | QGT trace: 0.0000+0.0000j | QGT condition: 2.69e+68 训练进度: 82%|▉▉▉▉▉▉▉▉▉▉▉▉▉ | 653/800 [00:28<00:06, 23.06it/s]

Iter 650 | Energy: -0.94098489 Ha | Error: 0.074483 Ha | QGT trace: 0.1300+0.0000j | QGT condition: 6.63e+47 训练进度: 88%|▉▉▉▉▉▉▉▉▉▉▉▉▉▉ | 704/800 [00:30<00:04, 23.30it/s]

Iter 700 | Energy: -0.94130673 Ha | Error: 0.074162 Ha | QGT trace: 0.0000+0.0000j | QGT condition: 2.58e+65 训练进度: 94%|▉▉▉▉▉▉▉▉▉▉▉▉▉▉▉ | 755/800 [00:32<00:01, 23.37it/s]

Iter 750 | Energy: -0.94132098 Ha | Error: 0.074147 Ha | QGT trace: 0.0000+0.0000j | QGT condition: 1.62e+65 训练进度: 100%|▉▉▉▉▉▉▉▉▉▉▉▉▉▉▉▉| 800/800 [00:34<00:00, 23.17it/s]

5 = ==========

训练完成！ 最终能量: -0.94134622 HA FCI 基准: -1.01546825 HA 误差: 7.412202E-02 HA

`

使用原生 Netket 的 VMC API 来实现的话:

```
# NetKet 原生实现
rngs = nnx.Rngs(21)
model_netket = SingleStateAnsatz(n_spin_orbitals=4, hidden_dim=12, rngs=rngs)

# 设置采样器
edges = [(0, 1), (2, 3)]
g = nk.graph.Graph(edges=edges)
single_rule = nk.sampler.rules.FermionHopRule(hilbert=hi, graph=g)
sampler = nk.sampler.MetropolisSampler(hi, rule=single_rule, n_chains=16, sweep_size=32)

# 创建 MCState
vstate = nk.vqs.MCState(
    sampler=sampler,
    model=model_netket,
    n_samples=1000,
    n_discard_per_chain=10
)

# 设置 SR 和优化器
preconditioner = nk.optimizer.SR(diag_shift=1e-3, holomorphic=True)
optimizer = nk.optimizer.Sgd(0.01)

# 创建 VMC 驱动
vmc = nk.driver.VMC(ha, optimizer, variational_state=vstate, preconditioner=preconditioner)

print(f"\n{'='*70}")
print("NetKet 原生实现训练")
print(f"{'='*70}\n")

# 使用 NetKet 的 run 方法
vmc.run(400, out=None)  # out=None 表示不保存到文件

# 获取最终能量
energy_final_netket = vstate.expect(ha)

print(f"\n{'='*70}")
print(f"NetKet 训练完成! 最终能量: {energy_final_netket.mean.real:.8f} Ha")
print(f"{'='*70}\n")
```

6 = 返回的结果是: `==========

NETKET 原生实现训练

100%|██████████| 300/300 [00:14<00:00, 20.61it/s, Energy=-1.015e+00-1.115e-09j ± 2.006e-10 [σ²=4.054e-17, R̂=1.3191]]

========== NetKet 训练完成！最终能量: -1.01546825 Ha ==========`

现在的问题是: 1. 我自己手动实现的方式 只能收敛到 HF 态能量，而不能收敛到 FCI 基准能量 2. 我给出的 Ansatz 没有问题 因为原生实现是成功的.